

PS26 - SE 02 Big Data & Analytics 2Sem A

Social media sentiment analysis regarding Ryanair

Team Project

My Contribution: Writing the Queries, querying the data, uploading the data to Tableau and creating visualizations.

03.07.2026

Table of Contents

1. Introduction	3
1.1 Project goal	3
1.2 Business value	3
1.3 Methodology	3
1.4 Outline	4
2. Data collection	4
2.1 Extracting the posts	5
2.2 Extracting the comments	6
2.3 Confirmation of successful extraction	7
3. Preprocessing	8
3.1 Reducer.py	9
3.2 Mapper.py	9
3.3 Successful mapping and reducing	10
4. Analysis	11
4.1 Loading the data into Postgres	11
4.2 Sentiment analysis using Python	13
4.3 Querying the data	14
4.4 Saving the results	16
5. Visualizations	16
5.1 Sentiment Distribution	17
5.2 Theme Summary	18
5.3 Time trend	19
6. Conclusions & Recommendations	20

1. Introduction

1.1 Project goal

The purpose of this project is to analyze social media sentiment regarding Ryanair. Through our analysis we will be able to identify public perception of the airline, and uncover key trends related to customer satisfaction, service quality, pricing, and travel experiences. With our insights we will have an overview over how customers view the brand Ryanair and we will be able to give recommendations for Ryanair on how they could improve their service.

1.2 Business value

The findings of this project will provide insights into how customers perceive Ryanair. By analyzing the sentiment shown in reddit posts and comments we will be able to identify Ryanair's strengths and weaknesses concerning their service, pricing and overall satisfaction. In this way Ryanair can address customer complaints and also be able to improve their services they offer to their customers. By addressing these things Ryanair can enhance the customer experience and possibly even attract new customers.

In general sentiment analyses can support decision-making in business because it helps Ryanair to monitor their brand reputation, identify trends in customer feedback and also determine the impact of any changes they make to their operations. In this way Ryanair will be able to figure out more effective marketing strategies and improve customer retention and customer loyalty.

1.3 Methodology

In order to collect data from reddit for our project we did not use API's as recommended but we used a different topic which we will explain in detail in the data collection section. The reason we chose a different method was because of the difficulties we faced while trying to use the reddit API. Our extracted data was then saved in a JSON format. For the preprocessing we used MapReduce to clean the data. After the cleaning we loaded the data into Hadoop for Processing and then used SQL Queries and Python VADER for sentiment classification. We stored the analysis results back into the database and then exported it to Tableau for visualization. In total we created three visualizations.

1.4 Outline

The outline of our report is as follows. Firstly we will present the details surrounding Data collection. We will talk about what methods we used to collect posts, comments and other data mentioning Ryanair and also how we stored the information. Afterwards we will show the preprocessing step. Here we will define how we cleaned the data and also what challenges came up during the cleaning process. The main part of our report will be the Analysis section. In this section we will summarize the findings we were able to draw from the data. These findings were extracted from the SQL Analysis and sentiment models. After the findings have been presented we will move on to our visualizations. We will include annotated screenshots of the Visualizations we created in this section. Lastly, we have our conclusion where we summarize our content and then provide practical insights for our brand Ryanair.

2. Data collection

For the data collection we weren't able to use the recommended reddit API as we had difficulties accessing it. Using other social media platforms was also not an option for us as we wanted to use reddit data specifically. We thought that reddit data would give us more insight into customer perception and would just have more data to work with overall.

To get the reddit data we first logged into our reddit account and searched for the Ryanair subreddit using this keyword "r/Ryanair". After we had access to the subreddit the only we had to do was to convert it to an xml format to make it easier to be processed by python. To do that we added .rss to the link of the Ryanair subreddit in the search bar. So the link went from this: <https://www.reddit.com/r/Ryanair/> to this: <https://www.reddit.com/r/ryanair/new/.rss>. We now had the raw data to the After receiving the data we used two different python codes, with the request library, to extract the comments and the posts from the raw data using our terminal. The only downside to this method is that it only gives us data from a very small time range. In our case it was from the 4th of June 2026 until the 19th of June 2026. Even just downloading data from this timeframe took a lot of time. These are the python codes we used to extract the posts and comments relevant to us from the data.

2.1 Extracting the posts

```
import requests
import json
import time
import xml.etree.ElementTree as ET

headers = {
    "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
}

# Different angles to maximise posts collected
sources = [
    ("ryanair", "new"),
    ("ryanair", "hot"),
    ("ryanair", "top"),
    ("travel", "new"),
    ("flights", "new"),
    ("europe", "new"),
]

all_posts = []
seen_urls = set() # avoid duplicates

for sub, sort in sources:
    print(f"\nCollecting from r/{sub} ({sort})...")

    # search for "ryanair" keyword in broader subreddits
    if sub == "ryanair":
        url = f"https://www.reddit.com/r/{sub}/{sort}/.rss?limit=100"
    else:
        url = f"https://www.reddit.com/r/{sub}/search.rss?q=ryanair&sort={sort}&limit=100"

    try:
        resp = requests.get(url, headers=headers, timeout=10)
        print(f"Status: {resp.status_code}")

        if resp.status_code == 429:
            print("Rate limited. Waiting 30 seconds...")
            time.sleep(30)
            resp = requests.get(url, headers=headers, timeout=10)
            print(f"Retry status: {resp.status_code}")

        if resp.status_code != 200:
            print(f"Skipping r/{sub} ({sort})")
            time.sleep(5)
            continue

        root = ET.fromstring(resp.content)
        namespace = {"atom": "http://www.w3.org/2005/Atom"}
        entries = root.findall("atom:entry", namespace)

        new_count = 0
        for entry in entries:
            title = entry.find("atom:title", namespace)
            content = entry.find("atom:content", namespace)
            link = entry.find("atom:link", namespace)
            published = entry.find("atom:published", namespace)
            author = entry.find("./atom:name", namespace)

            post_url = link.get("href") if link is not None else ""

            # skip duplicates
            if post_url in seen_urls:
                continue
            seen_urls.add(post_url)

            all_posts.append({
                "subreddit": sub,
                "sort": sort,
                "title": title.text if title is not None else "",
                "text": content.text if content is not None else "",
                "url": post_url,
                "published": published.text if published is not None else "",
                "author": author.text if author is not None else "",
            })
            new_count += 1

        print(f"Got {new_count} new posts from r/{sub} ({sort})")
        print(f"Total so far: {len(all_posts)}")

    except Exception as e:
        print(f"Error on r/{sub} ({sort}): {e}")

    time.sleep(10) # polite delay between each source

with open("reddit_posts.json", "w") as f:
    json.dump(all_posts, f, indent=2)

print(f"\n Done! Saved {len(all_posts)} REAL current Reddit posts to reddit_posts.json")
```

2.2 Extracting the comments

```
import requests
import json
import time
import xml.etree.ElementTree as ET

headers = {
    "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
}

# Load existing posts
with open("reddit_posts.json", "r") as f:
    posts = json.load(f)

print(f"Loaded {len(posts)} posts. Now collecting comments...\n")

all_comments = []
failed = 0

for i, post in enumerate(posts):
    post_url = post.get("url", "")
    if not post_url:
        continue

    # Convert post URL to RSS comments feed
    rss_url = post_url.rstrip("/") + "/.rss"

    try:
        resp = requests.get(rss_url, headers=headers, timeout=10)

        if resp.status_code == 429:
            print(f"[{i+1}/{len(posts)}] Rate limited. Waiting 30s...")
            time.sleep(30)
            resp = requests.get(rss_url, headers=headers, timeout=10)

        if resp.status_code != 200:
            failed += 1
            time.sleep(3)
            continue

        root = ET.fromstring(resp.content)
        namespace = {"atom": "http://www.w3.org/2005/Atom"}
        entries = root.findall("atom:entry", namespace)

        post_comments = 0
        for entry in entries:
            title = entry.find("atom:title", namespace)
            content = entry.find("atom:content", namespace)
            published = entry.find("atom:published", namespace)
            author = entry.find("./atom:name", namespace)

            comment_text = content.text if content is not None else ""
            if not comment_text or len(comment_text.strip()) < 5:
                continue

            all_comments.append({
                "post_title": post.get("title", ""),
                "post_subreddit": post.get("subreddit", ""),
                "comment": comment_text,
                "published": published.text if published is not None else "",
                "author": author.text if author is not None else "",
                "post_url": post_url,
            })
            post_comments += 1

        print(f"[{i+1}/{len(posts)}] Got {post_comments} comments | Total: {len(all_comments)}")

    except Exception as e:
        print(f"[{i+1}/{len(posts)}] Error: {e}")
        failed += 1

    time.sleep(2) # polite delay per post

with open("reddit_comments.json", "w") as f:
    json.dump(all_comments, f, indent=2)

print(f"\n Done!")
print(f" Comments collected : {len(all_comments)}")
print(f" Posts failed       : {failed}")
print(f" Saved to             : reddit_comments.json")
```

2.3 Confirmation of successful extraction

```
[173/173] Rate limited. Waiting 30s...  
[173/173] Got 1 comments | Total: 1382
```

Done!

Comments collected : 1382

Posts failed : 79

Saved to : reddit_comments.json

(venv) munyendo@Lauras-MacBook-Pro ryanair_project %

```
Retry status: 200  
Got 0 new posts from r/flights (new)  
Total so far: 173
```

```
Collecting from r/europe (new)...  
Status: 429  
Rate limited. Waiting 30 seconds...  
Retry status: 429  
Skipping r/europe (new)
```

Done! Saved 173 REAL current Reddit posts to reddit_posts.json

3. Preprocessing

Our Data Cleaning stage began with us loading the data into Hadoop. We first logged into our Hadoop account and used the `hdfs dfs -put -f` command with the path to the files to add the reddit comments and posts data into the input directory on Hadoop. To confirm that the data was really added we used the `hdfs dfs -ls` command to see what files we have in our input directory. As expected our reddit comments and posts files showed up.

```
hadoop_user@LAURA-UBUNTU:~$ hdfs dfs -put -f ~/reddit_posts.json /ryanair/input/
2026-06-20 18:20:35,461 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
hadoop_user@LAURA-UBUNTU:~$ hdfs dfs -put -f ~/reddit_comments.json /ryanair/input/
2026-06-20 18:21:50,528 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
put: `~/reddit_comments.json': No such file or directory
hadoop_user@LAURA-UBUNTU:~$ hdfs dfs -put -f ~/reddit_comments.json /ryanair/input/
2026-06-20 18:23:16,461 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
hadoop_user@LAURA-UBUNTU:~$ hdfs dfs -ls /ryanair/input
2026-06-20 18:26:40,452 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r--  1 hadoop_user supergroup      962538 2026-06-20 18:23 /ryanair/input/reddit_comm
ents.json
-rw-r--r--  1 hadoop_user supergroup      286893 2026-06-20 18:20 /ryanair/input/reddit_post
s.json
```

When trying to clean up our data using the json files on Hadoop we encountered a lot of errors. To fix those errors we had to convert both json files into NDJSON files. NDJSON files are better for Hadoop to read because in a JSON file a single array of objects is stored while an NDJSON file separates single JSON objects by newline characters. After we converted the files we were able to clean our data using Mapreduce.

```
hadoop_user@LAURA-UBUNTU:~/ryanair_local$ wc -l reddit_posts.ndjson reddit_comments.ndjson
  173 reddit_posts.ndjson
 1382 reddit_comments.ndjson
 1555 total
hadoop_user@LAURA-UBUNTU:~/ryanair_local$ 
hadoop_user@LAURA-UBUNTU:~/ryanair_local$ hdfs dfs -put -f reddit_posts.ndjson /ryanair/inpu
t/
2026-06-20 18:43:47,918 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
hadoop_user@LAURA-UBUNTU:~/ryanair_local$ hdfs dfs -put -f reddit_comments.ndjson /ryanair/i
nput/
2026-06-20 18:44:10,051 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
hadoop_user@LAURA-UBUNTU:~/ryanair_local$ hdfs dfs -ls /ryanair/input
2026-06-20 18:44:28,729 WARN util.NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
Found 4 items
-rw-r--r--  1 hadoop_user supergroup      962538 2026-06-20 18:23 /ryanair/input/reddit_comm
ents.json
-rw-r--r--  1 hadoop_user supergroup      912588 2026-06-20 18:44 /ryanair/input/reddit_comm
ents.ndjson
-rw-r--r--  1 hadoop_user supergroup      286893 2026-06-20 18:20 /ryanair/input/reddit_post
s.json
-rw-r--r--  1 hadoop_user supergroup      277962 2026-06-20 18:43 /ryanair/input/reddit_post
s.ndjson
```

3.1 Reducer.py

```
#!/usr/bin/env python3
import sys

# Pass-through reducer: we just emit cleaned rows.
# (You can later add aggregation here if needed.)
for line in sys.stdin:
    line = line.rstrip("\n")
    if line:
        print(line)
```

3.2 Mapper.py

```
#!/usr/bin/env python3
import sys
import json
import re
import html

url_re = re.compile(r"http\S+|www\.\S+")
tag_re = re.compile(r"<.*?>")

def clean_text(s: str) -> str:
    s = html.unescape(s)
    s = url_re.sub(" ", s)
    s = tag_re.sub(" ", s)
    s = s.replace("\n", " ").replace("\r", " ")
    s = re.sub(r"\s+", " ", s).strip().lower()
    return s

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue

    # Each line should be one JSON object (NDJSON)
    try:
        rec = json.loads(line)
    except Exception:
        continue

    # Identify record type and fields
    # Posts file has: title, text, published, subreddit, url
    # Comments file has: comment, published, post_subreddit, post_url (and sometimes post_title)
    if "comment" in rec:
        record_type = "comment"
        subreddit = rec.get("post_subreddit") or rec.get("subreddit") or "unknown"
        url = rec.get("post_url") or rec.get("url") or ""
        published = rec.get("published") or ""
        base_text = " ".join([
            rec.get("post_title") or "",
            rec.get("comment") or ""
        ])
    else:
        record_type = "post"
        subreddit = rec.get("subreddit") or "unknown"
        url = rec.get("url") or ""
        published = rec.get("published") or ""
        base_text = " ".join([
            rec.get("title") or "",
            rec.get("text") or ""
        ])

    cleaned = clean_text(base_text)

    # filter out very short junk
    if len(cleaned) < 15:
        continue

    # Output: type, subreddit, published, url, text
    print(f"{record_type}\t{subreddit}\t{published}\t{url}\t{cleaned}")
```

3.3 Successful mapping and reducing

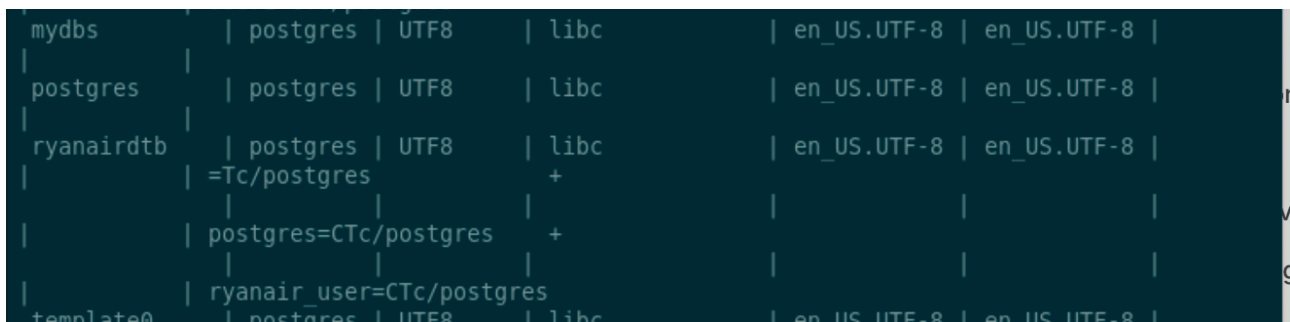
```
hadoop_user@LAURA-UBUNTU:~/ryanair_local$ hadoop jar ~/hadoop-3.4.3/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-D mapreduce.job.name="ryanair-cleaning" \
-input /ryanair/input/reddit_posts.ndjson \
-input /ryanair/input/reddit_comments.ndjson \
-output /ryanair/output/cleaned \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-file ~/ryanair_project/mapper.py \
-file ~/ryanair_project/reducer.py
2026-06-20 19:23:27,053 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
2026-06-20 19:23:27,155 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
packageJobJar: [/home/hadoop_user/ryanair_project/mapper.py, /home/hadoop_user/ryanair_project/reducer.py, /tmp/hadoop-unjar8952428630545593380/] [] /tmp/streamjob7145432409671590226.jar tmpDir=null
2026-06-20 19:23:27,762 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /127.0.0.1:8032
2026-06-20 19:23:27,944 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /127.0.0.1:8032
2026-06-20 19:23:29,478 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop_user/.staging/job_1781965812849_0001
2026-06-20 19:23:30,472 INFO mapred.FileInputFormat: Total input files to process : 2
2026-06-20 19:23:30,550 INFO mapreduce.JobSubmitter: number of splits:3
2026-06-20 19:23:30,718 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1781965812849_0001
2026-06-20 19:23:30,718 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-06-20 19:23:31,007 INFO conf.Configuration: resource-types.xml not found
2026-06-20 19:23:31,007 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-06-20 19:23:32,931 INFO impl.YarnClientImpl: Submitted application application_1781965812849_0001
2026-06-20 19:23:33,049 INFO mapreduce.Job: The url to track the job: http://LAURA-UBUNTU:8088/proxy/application_1781965812849_0001/
2026-06-20 19:23:33,053 INFO mapreduce.Job: Running job: job_1781965812849_0001
2026-06-20 19:23:46,776 INFO mapreduce.Job: Job job_1781965812849_0001 running in uber mode : false
2026-06-20 19:23:46,790 INFO mapreduce.Job:  map 0% reduce 0%
2026-06-20 19:23:54,241 INFO mapreduce.Job:  map 100% reduce 0%
2026-06-20 19:24:01,402 INFO mapreduce.Job:  map 100% reduce 100%
2026-06-20 19:24:03,541 INFO mapreduce.Job: Job job_1781965812849_0001 completed successfully
y
```

4. Analysis

The analysis part in our project was divided into two parts. First we did the SQL Analysis using our database and different SQL Queries and then afterwards we did the Sentiment analysis with python in order to figure the overall sentiment distribution (neutral, negative, positive).

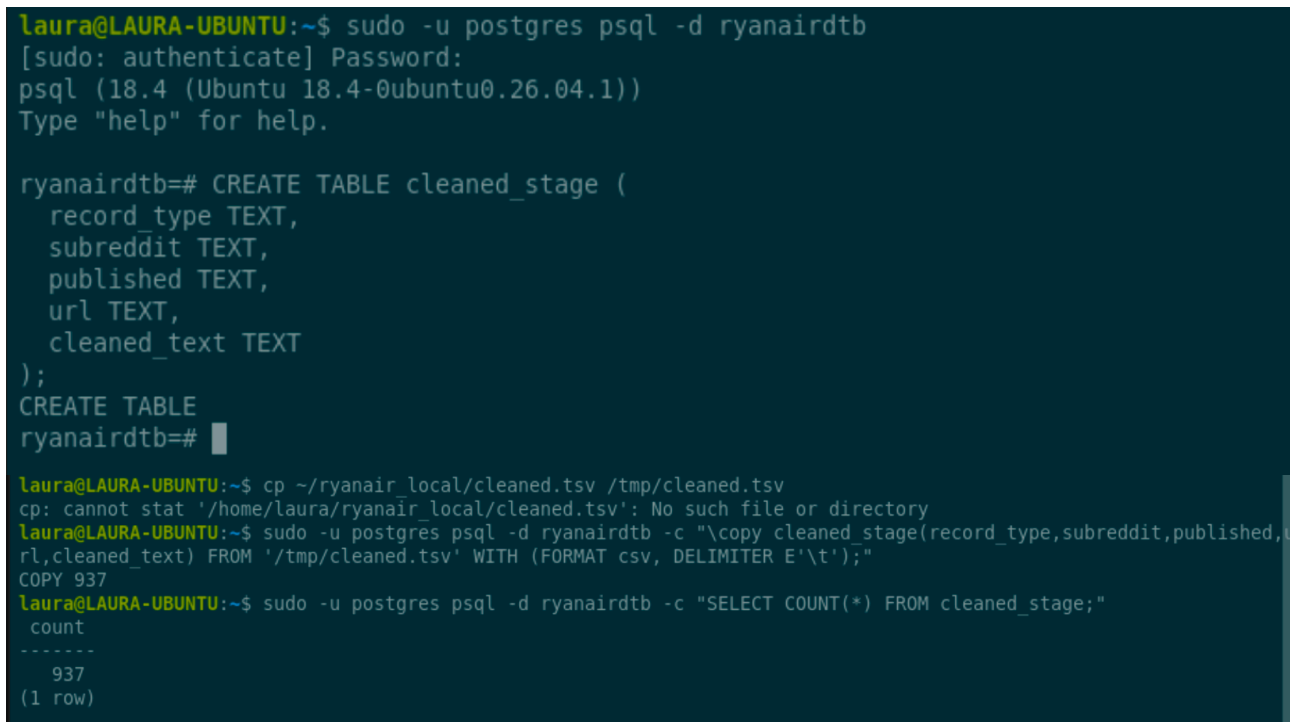
4.1 Loading the data into Postgres

Before writing any queries we first had to create a database in Postgres. To do that we typed "CREATE DATABASE ryanairdtb" into the terminal on our Virtual machine. After we created the database we used "\l" to confirm the creation of our database.



mydbs	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8
postgres	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8
ryanairdtb	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8
	=Tc/postgres		+		
	postgres=CTc/postgres		+		
	ryanair_user=CTc/postgres				
template0	postgres	UTF8	libc	en_US.UTF-8	en_US.UTF-8

After creating the database we created two tables. The first one was called cleaned_stage. In this table we inserted the data we received as an output from the data cleaning step.



```
laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdtb
[sudo: authenticate] Password:
psql (18.4 (Ubuntu 18.4-0ubuntu0.26.04.1))
Type "help" for help.

ryanairdtb=# CREATE TABLE cleaned_stage (
  record_type TEXT,
  subreddit TEXT,
  published TEXT,
  url TEXT,
  cleaned_text TEXT
);
CREATE TABLE
ryanairdtb=#
```

```
laura@LAURA-UBUNTU:~$ cp ~/ryanair_local/cleaned.tsv /tmp/cleaned.tsv
cp: cannot stat '/home/laura/ryanair_local/cleaned.tsv': No such file or directory
laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdtb -c "\copy cleaned_stage(record_type,subreddit,published,
url,cleaned_text) FROM '/tmp/cleaned.tsv' WITH (FORMAT csv, DELIMITER E'\t');"
COPY 937
laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdtb -c "SELECT COUNT(*) FROM cleaned_stage;"
count
-----
937
(1 row)
```

The last command was added by us to confirm the number of rows of the cleaned_stage table, so that we were sure that everything was transferred.

The second table was called `reddit_data`. This was the table we actually going to query and do the analysis with. The first table only served as some kind of back-up in case anything goes wrong with querying and we lose access to our data. By doing it in this way if something happens we will always have the `cleaned_stage` table with all of our original data to look back on.

```
ryanairdtb=# \reset
Query buffer reset (cleared).
ryanairdtb=# CREATE TABLE reddit_data (
    id SERIAL PRIMARY KEY,
    subreddit TEXT,
    title TEXT,
    body TEXT,
    comment TEXT,
    published TEXT,
    author TEXT,
    url TEXT,
    sentiment TEXT,
    sentiment_score FLOAT);
CREATE TABLE
ryanairdtb=# \dt
          List of tables
 Schema |   Name   | Type | Owner
-----+-----+-----+-----
 public | reddit_data | table | postgres
(1 row)

ryanairdtb=#
```

To insert the data from the `cleaned_stage` table to the `reddit_data` table we used `INSERT INTO` and specified what data exactly we wanted to be inserted. We also made a distinction between posts and comments by redefining the `cleaned_text` column from the table `cleaned_stage` either as `body` or `comment` depending on if the `record_type` is `post` or `comment`. To confirm that everything was inserted correctly we checked the status using the queries in the last two lines.

```
laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdtb -c "
INSERT INTO reddit_data (subreddit, body, comment, published, url)
SELECT
    subreddit,
    CASE WHEN record_type='post' THEN cleaned_text ELSE NULL END AS body,
    CASE WHEN record_type='comment' THEN cleaned_text ELSE NULL END AS comment,
    published,
    url
FROM cleaned_stage;
"
INSERT 0 937
laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdtb -c "SELECT COUNT(*) AS total_rows FROM reddit_data;"
total_rows
-----
937
(1 row)

laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdtb -c "SELECT COUNT(body) AS post_rows, COUNT(comment) AS co
mment_rows FROM reddit data;"
post_rows | comment_rows
-----+-----
90 | 847
(1 row)
```

4.2 Sentiment analysis using Python

Before we were able to do the sentiment analysis using python we first had to install pip. After installing that we were then able to install pandas, psycopg2-binary and then vaderSentiment for the analysis.

```
laura@LAURA-UBUNTU:~$ cd ~/ryanair_sentiment
laura@LAURA-UBUNTU:~/ryanair_sentiment$ python3 -m venv .venv
laura@LAURA-UBUNTU:~/ryanair_sentiment$ source .venv/bin/activate
(.venv) laura@LAURA-UBUNTU:~/ryanair_sentiment$ python -m pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.14/site-packages (25.1.1)
Collecting pip
  Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 27.5 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
  Successfully installed pip-26.1.2
(.venv) laura@LAURA-UBUNTU:~/ryanair_sentiment$ pip install pandas psycopg2-binary vaderSentiment
Collecting pandas
  Downloading pandas-3.0.3-cp314-cp314-manylinux_2_24_aarch64.manylinux_2_28_aarch64.whl.metadata (79 kB)
Collecting psycopg2-binary
  Downloading psycopg2-binary-2.9.12-cp314-cp314-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.metadata (4.9 kB)
```

To do the sentiment analysis with python we used this code:

```
import pandas as pd
import psycopg2
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

DB_NAME = "ryanairdb"
DB_USER = "ryanair_user"
DB_PASS = "ryanair123"
DB_HOST = "localhost"
DB_PORT = 5432

analyzer = SentimentIntensityAnalyzer()

def label(compound: float) -> str:
    if compound >= 0.05:
        return "positive"
    if compound <= -0.05:
        return "negative"
    return "neutral"

conn = psycopg2.connect(
    dbname=DB_NAME,
    user=DB_USER,
    password=DB_PASS,
    host=DB_HOST,
    port=DB_PORT
)

df = pd.read_sql_query(
    """
    SELECT id, COALESCE(comment, body) AS text
    FROM reddit_data
    WHERE COALESCE(comment, body) IS NOT NULL
    AND sentiment_score IS NULL
    """,
    conn
)

print("Rows to score:", len(df))

scores = []
labels = []
for t in df["text"].astype(str):
    comp = analyzer.polarity_scores(t)["compound"]
    scores.append(comp)
    labels.append(label(comp))

df["sentiment_score"] = scores
df["sentiment"] = labels

cur = conn.cursor()
for row in df.iteruples(index=False):
    cur.execute(
        "UPDATE reddit_data SET sentiment=%, sentiment_score=%s WHERE id=%s",
        (row.sentiment, float(row.sentiment_score), int(row.id))
    )
```

```

conn.commit()
cur.close()
conn.close()

print("Done. Updated:", len(df))

```

```

(.venv) laura@LAURA-UBUNTU:~$ cd ~/ryanair_sentiment
(.venv) laura@LAURA-UBUNTU:~/ryanair_sentiment$ source .venv/bin/activate
(.venv) laura@LAURA-UBUNTU:~/ryanair_sentiment$ python sentiment.py
/home/laura/ryanair_sentiment/sentiment.py:28: UserWarning: pandas only supports SQLAlchemy connectable (engine/con
nection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider
using SQLAlchemy.
  df = pd.read_sql_query(
Rows to score: 937
Done. Updated: 937
(.venv) laura@LAURA-UBUNTU:~/ryanair_sentiment$ █

```

4.3 Querying the data

The first query we wrote was regarding the overall sentiment distribution

```

(.venv) laura@LAURA-UBUNTU:~/ryanair_sentiment$ cd ~
(.venv) laura@LAURA-UBUNTU:~$ mkdir -p ~/ryanair_results
(.venv) laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdb -c "\copy (SELECT sent
iment, COUNT(*) AS count FROM reddit data GROUP BY sentiment ORDER BY count DESC) TO '/t
mp/sentiment_distribution.csv' CSV HEADER"
COPY 3

```

The second query was regarding the theme summary

```

laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdb <<'SQL'
COPY (
  SELECT
    theme,
    COUNT(*) AS count,
    AVG(sentiment_score) AS avg_sentiment
  FROM (
    SELECT
      CASE
        WHEN COALESCE(comment, body) ILIKE '%bag%'
        OR COALESCE(comment, body) ILIKE '%baggage%'
        OR COALESCE(comment, body) ILIKE '%luggage%'
        OR COALESCE(comment, body) ILIKE '%personal item%'
        OR COALESCE(comment, body) ILIKE '%cabin bag%'
        OR COALESCE(comment, body) ILIKE '%underseat%'
        OR COALESCE(comment, body) ILIKE '%carry on%'
        OR COALESCE(comment, body) ILIKE '%carry-on%'
        THEN 'baggage'

        WHEN COALESCE(comment, body) ILIKE '%boarding pass%'
        OR COALESCE(comment, body) ILIKE '%check in%'
        OR COALESCE(comment, body) ILIKE '%check-in%'
        OR COALESCE(comment, body) ILIKE '%checked in%'
        OR COALESCE(comment, body) ILIKE '%digital boarding%'
        OR COALESCE(comment, body) ILIKE '%rejected%'
        THEN 'check-in/boarding'

        WHEN COALESCE(comment, body) ILIKE '%visa%'
        OR COALESCE(comment, body) ILIKE '%passport%'
        OR COALESCE(comment, body) ILIKE '%eta%'
        OR COALESCE(comment, body) ILIKE '%document%'
        OR COALESCE(comment, body) ILIKE '%id card%'
        OR COALESCE(comment, body) ILIKE '%residence permit%'
        THEN 'travel documents'

        WHEN COALESCE(comment, body) ILIKE '%delay%'
        OR COALESCE(comment, body) ILIKE '%delayed%'
        OR COALESCE(comment, body) ILIKE '%late%'
        THEN 'delays'

        WHEN COALESCE(comment, body) ILIKE '%cancel%'
        OR COALESCE(comment, body) ILIKE '%cancelled%'
        OR COALESCE(comment, body) ILIKE '%canceled%'
        OR COALESCE(comment, body) ILIKE '%annullierung%'
        THEN 'cancellations'
      END
    )

```

```

        WHEN COALESCE(comment, body) ILIKE '%refund%'
        OR COALESCE(comment, body) ILIKE '%compensation%'
        OR COALESCE(comment, body) ILIKE '%claim%'
        OR COALESCE(comment, body) ILIKE '%customer service%'
        OR COALESCE(comment, body) ILIKE '%support%'
        THEN 'service/refunds'

        WHEN COALESCE(comment, body) ILIKE '%price%'
        OR COALESCE(comment, body) ILIKE '%prices%'
        OR COALESCE(comment, body) ILIKE '%fee%'
        OR COALESCE(comment, body) ILIKE '%charge%'
        OR COALESCE(comment, body) ILIKE '%cost%'
        OR COALESCE(comment, body) ILIKE '%paid%'
        THEN 'pricing/fees'

        WHEN COALESCE(comment, body) ILIKE '%connection%'
        OR COALESCE(comment, body) ILIKE '%connecting%'
        OR COALESCE(comment, body) ILIKE '%transfer%'
        OR COALESCE(comment, body) ILIKE '%separate ticket%'
        OR COALESCE(comment, body) ILIKE '%separate tickets%'
        THEN 'connections/transfers'

        WHEN COALESCE(comment, body) ILIKE '%gate%'
        OR COALESCE(comment, body) ILIKE '%security%'
        OR COALESCE(comment, body) ILIKE '%airport%'
        OR COALESCE(comment, body) ILIKE '%customs%'
        OR COALESCE(comment, body) ILIKE '%tsa%'
        THEN 'airport/gate issues'

        WHEN COALESCE(comment, body) ILIKE '%app%'
        OR COALESCE(comment, body) ILIKE '%booking%'
        OR COALESCE(comment, body) ILIKE '%reservation%'
        OR COALESCE(comment, body) ILIKE '%confirmation email%'
        OR COALESCE(comment, body) ILIKE '%name change%'
        OR COALESCE(comment, body) ILIKE '%typo%'
        THEN 'app/booking issues'

        WHEN COALESCE(comment, body) ILIKE '%excellent%'
        OR COALESCE(comment, body) ILIKE '%positive experience%'
        OR COALESCE(comment, body) ILIKE '%pleasant%'
        OR COALESCE(comment, body) ILIKE '%no complaints%'
        OR COALESCE(comment, body) ILIKE '%would choose ryanair again%'
        THEN 'positive experience'

        ELSE 'uncategorized'
    END AS theme,
    sentiment_score
FROM reddit_data
WHERE sentiment_score IS NOT NULL
) AS themed_data
GROUP BY theme
ORDER BY count DESC
) TO '/tmp/theme_summary_refined.csv' CSV HEADER;
SQL

```

The last query was regarding the time trend:

```

laura@LAURA-UBUNTU:~$ sudo -u postgres psql -d ryanairdb <<'SQL'
COPY (
    SELECT
        DATE(published::timestampz) AS day,
        sentiment,
        COUNT(*) AS count
    FROM reddit_data
    WHERE sentiment IS NOT NULL
        AND published IS NOT NULL
        AND published <> ''
    GROUP BY day, sentiment
    ORDER BY day, sentiment
) TO '/tmp/time_trend.csv' CSV HEADER;
SQL
[sudo: authenticate] Password:
COPY 42

```

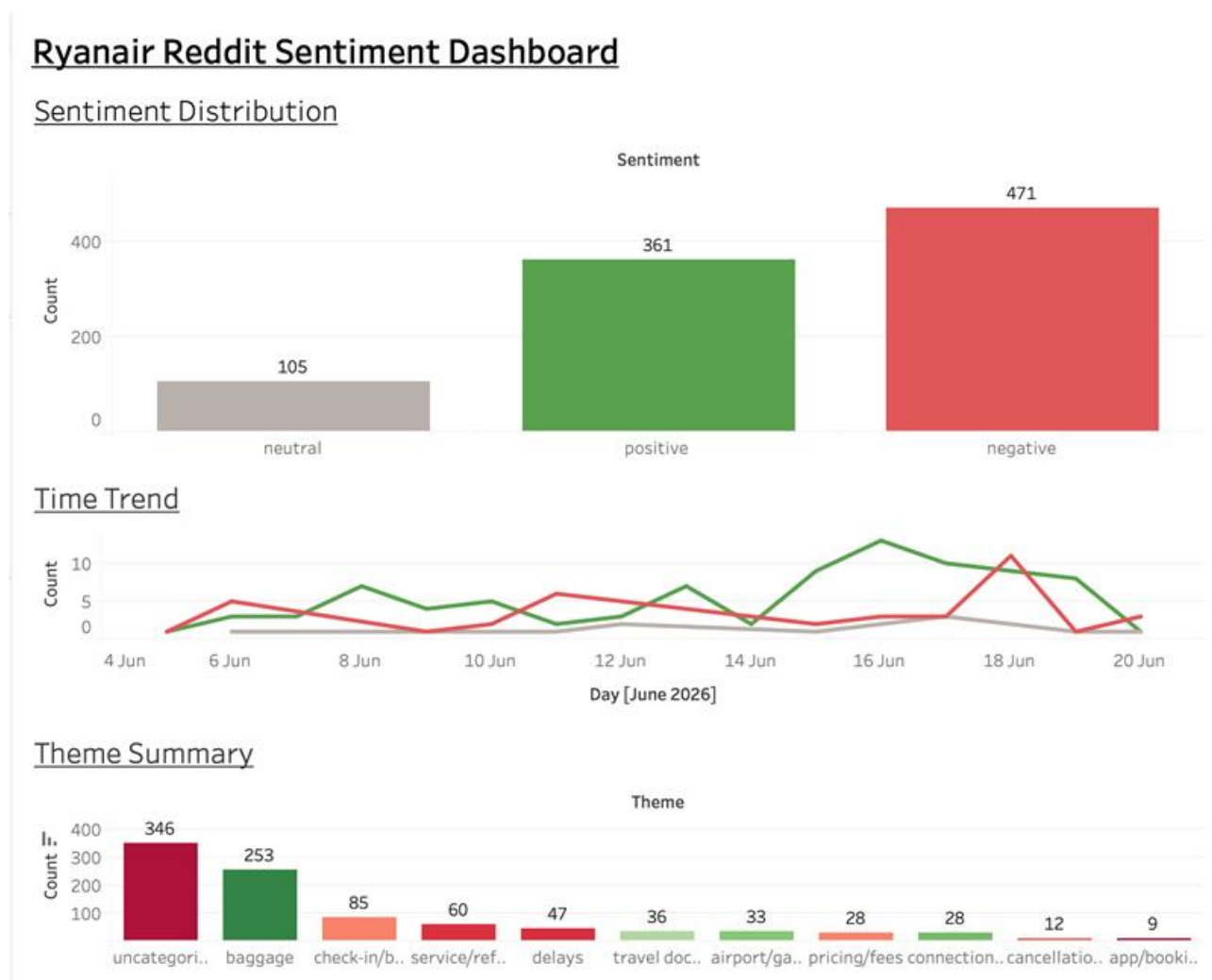
4.4 Saving the results

In order to save our results we copied it into a folder by saving it as a CSV file.

```
(.venv) laura@LAURA-UBUNTU:~$ cp /tmp/sentiment_distribution.csv ~/ryanair_results/
(.venv) laura@LAURA-UBUNTU:~$ cp /tmp/theme_summary.csv ~/ryanair_results/
(.venv) laura@LAURA-UBUNTU:~$ cp /tmp/subreddit_breakdown.csv ~/ryanair_results/
(.venv) laura@LAURA-UBUNTU:~$ ls -lh ~/ryanair_results
total 16K
-rw-r--r-- 1 laura laura 54 Jun 29 07:24 sentiment_distribution.csv
-rw-r--r-- 1 laura laura 94 Jun 29 07:24 subreddit_breakdown.csv
-rw-r--r-- 1 laura laura 218 Jun 29 07:24 theme_summary.csv
-rw-r--r-- 1 laura laura 26 Jun 29 07:19 time_trend.csv
(.venv) laura@LAURA-UBUNTU:~$
```

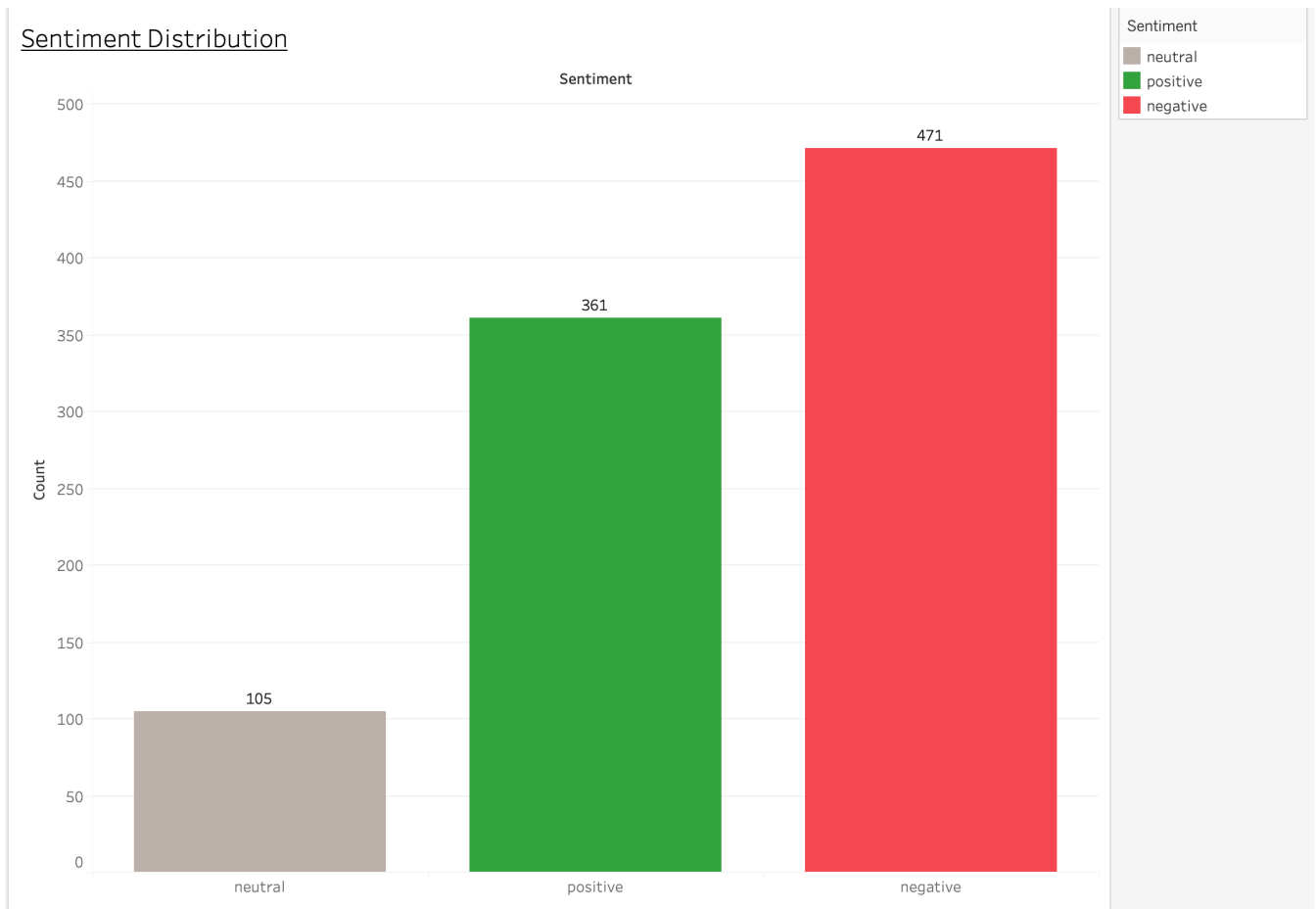
5. Visualizations

We created three visualizations from the data we extracted using Queries and sentiment models on Tableau. This is an overview of our dashboard.



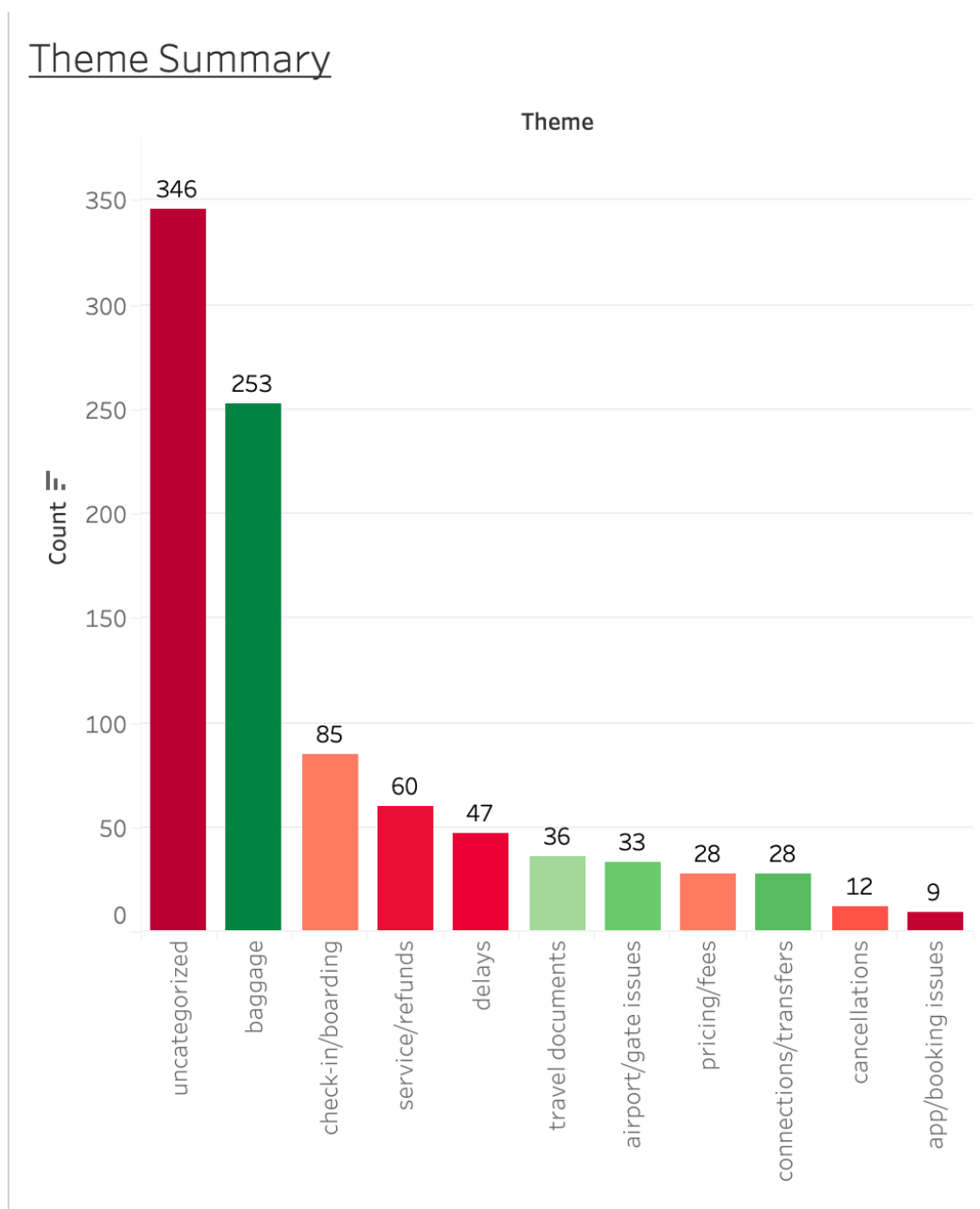
5.1 Sentiment Distribution

The first visualization we created was regarding the overall sentiment distribution of the reddit data. This visualization shows that 105 of the extracted data surrounding the Ryanair sentiment was neutral, 361 were positive and 471 were negative. This shows that the overall sentiment regarding Ryanair is negative.



5.2 Theme Summary

In the second visualization we go into more detail regarding the sentiment towards Ryanair and represent the themes of the comments and posts in a graphical manner to see what exactly was expressed by the users on reddit. The themes we were able to extract are baggage, check-in/boarding, service/ refunds, delays, travel documents, airport/gate issues, pricing/fees, connections/ transfers, cancellations and app/booking issues. The rest of the themes were uncategorized.

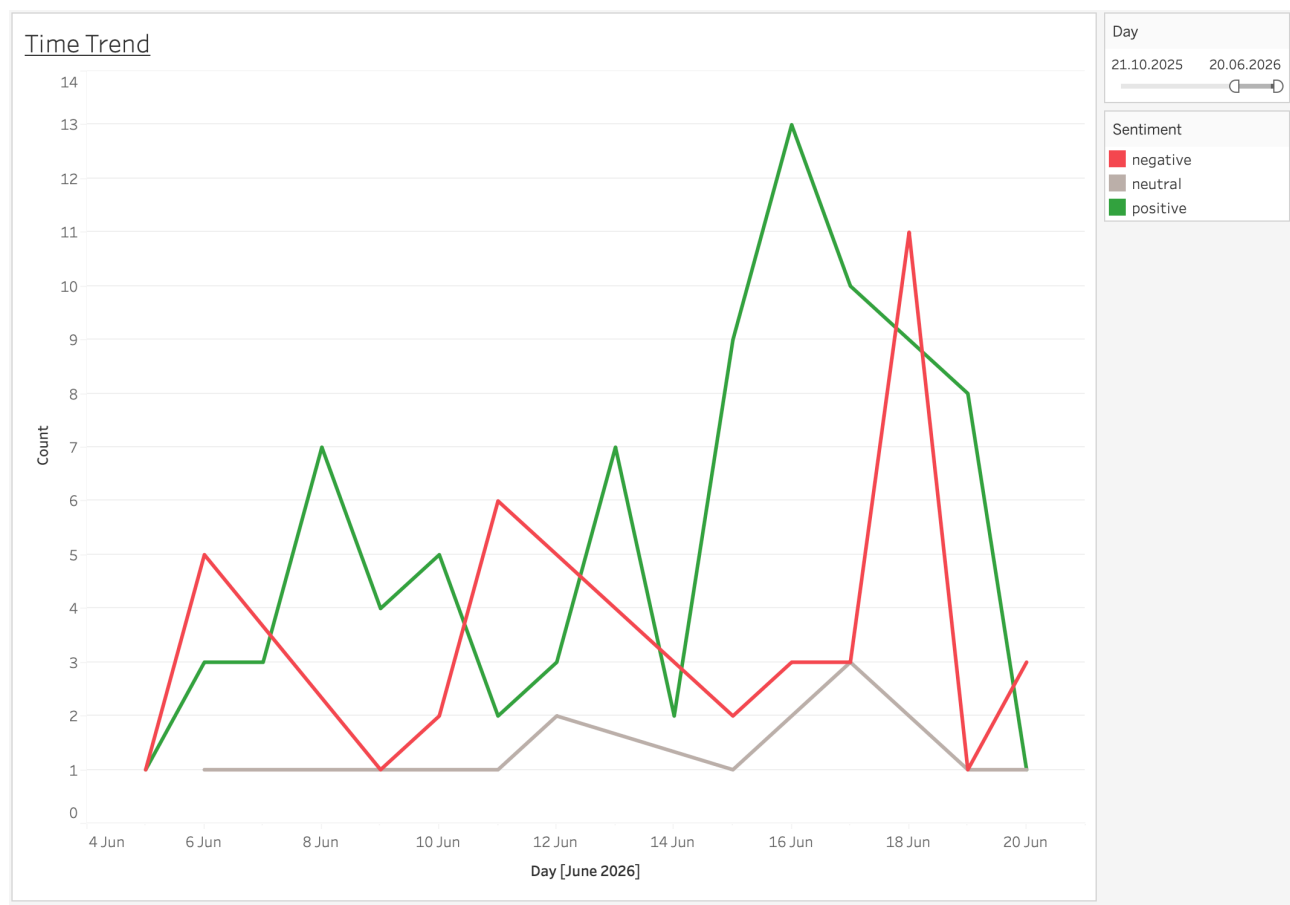


The colors in the bar chart represent whether this specific theme has a negative or positive sentiment. The bars which are dark red have a very strong negative sentiment,

those who are light red to orange have a mildly negative sentiment. The same goes for the green bars. Very dark green bars have a very positive sentiment and light green bars have a mildly positive sentiment. Out of all themes the only one with a very positive sentiment is baggage (253). Airport/gate issues and connections/transfers also have a positive sentiment but it is not as strong as the baggage theme. The theme with the least positive sentiment is the travel documents theme. In total 350 of the extracted data had a positive sentiment. Everything else had a negative sentiment. In the visualization we can see the majority of the negative sentiment regarding Ryanair comes from uncategorized data (346). App/booking issues, delays and service/refunds all have a stronger negative sentiment than check-in/boarding and pricing/fees. In total 587 of the extracted data had a negative sentiment. This shows that more than half of the data extracted had a positive sentiment.

5.3 Time trend

The last visualization we created was to show the sentiment over a specific time range.



The time range that is represented in this visualization is from the 4th of June until the 20th of June. It shows the amount of data in that time range that had a negative, positive

or neutral sentiment. As we can see there are drastic fluctuations in the distribution of the negative and positive sentiment over time. The neutral sentiment also has some fluctuations but they aren't as drastic as positive and negative. It is also important to note that between the 14th of June and the 16th of June the positive sentiment has a drastic spike. The negative sentiment also experiences a drastic spike but only between the 16th and 18th of June.

6. Conclusions & Recommendations

In conclusion we can say that it is very apparent that the overall sentiment regarding Ryanair is negative. The main sources of dissatisfaction are the app/booking issues, delays and service/refunds, check-in/boarding and pricing/fees as explained before in the visualizations section. These themes contributed to the negative sentiment surrounding Ryanair and show that customers are not satisfied with Ryanair's operational reliability, prices and booking process. On the other hand customers were very satisfied with Ryanair's baggage policies, transfers and airport procedures. Their main operations regarding flights and baggage options are the main reasons Ryanair is chosen over other airlines by a multitude of people.

Based on these findings, we have concluded that Ryanair should keep its baggage policies the same as they had a very strong positive sentiment. The main airport operations can also be kept the same but here it would be good to go into more detail regarding the data to find out why exactly the positive sentiment wasn't as strong as with the baggage theme. On the other hand, they need to make a lot of improvements in their overall service, when handling delays and the efficiency regarding their refund process. Another point to address would be the multitude of the uncategorized data that has a negative sentiment. Detailed investigation will be needed to find out what exactly causes the negative sentiment. If they address these issues they would be able to enhance customer satisfaction, improve the brand online reputation and also strengthen customer loyalty. By addressing the problems they could also attract customers that usually use other airlines as they have possibly heard about the negative sentiment regarding Ryanair.

The time trend insights also offer a chance for Ryanair. In the time-ranges where there was a drastic spike in the positive and negative sentiment a detailed investigation will be needed. For these time ranges Ryanair can figure out what exactly led to that spike. They can then determine if it was a special action they took that led to that outcome and adjust

their operations and campaigns accordingly. They could either limit or improve some operations that led to the negative sentiment or also keep the action that led to the positive sentiment.

Overall, this sentiment analysis shows how social media data can provide important business insights. If Ryanair continuously monitors customer sentiment on social media they will be able to identify any issues before they get too difficult to handle, make good decisions and improve its services and overall customer experience.